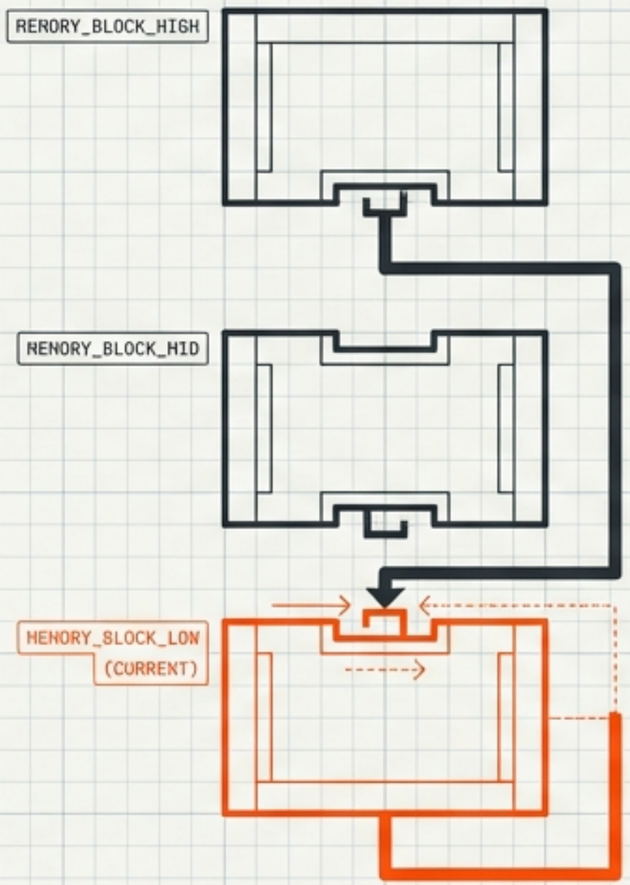


The Stack: Memory Layout & Execution Flow

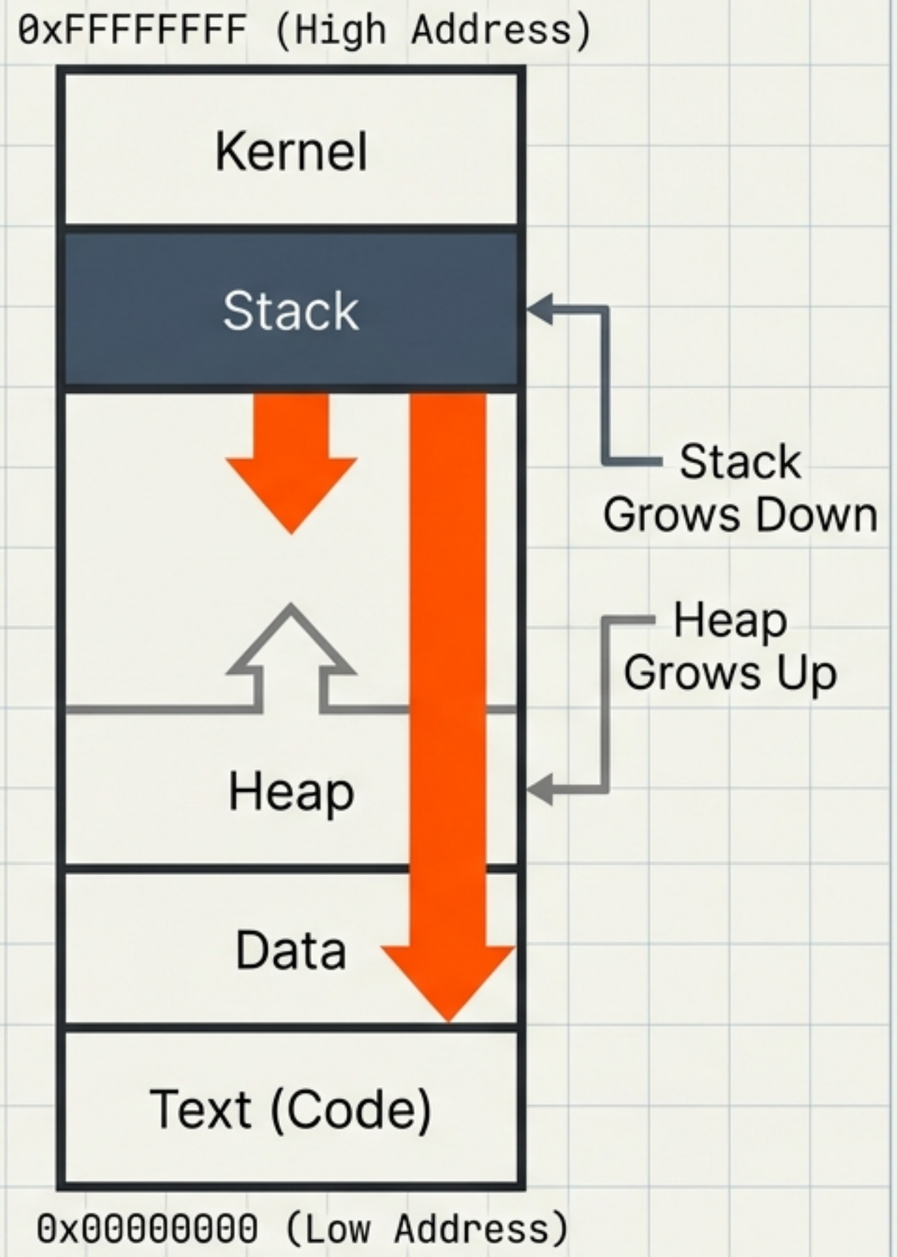
Visualizing x86 Architecture, Stack Frames, and the Function Call Lifecycle.

Based on the lecture '88 - The Stack' by Dr. Josh Stroschein

The 32-Bit Memory Map

Virtual memory is a linear address space. In x86 architecture, the Stack and Heap grow toward each other to maximize available space.

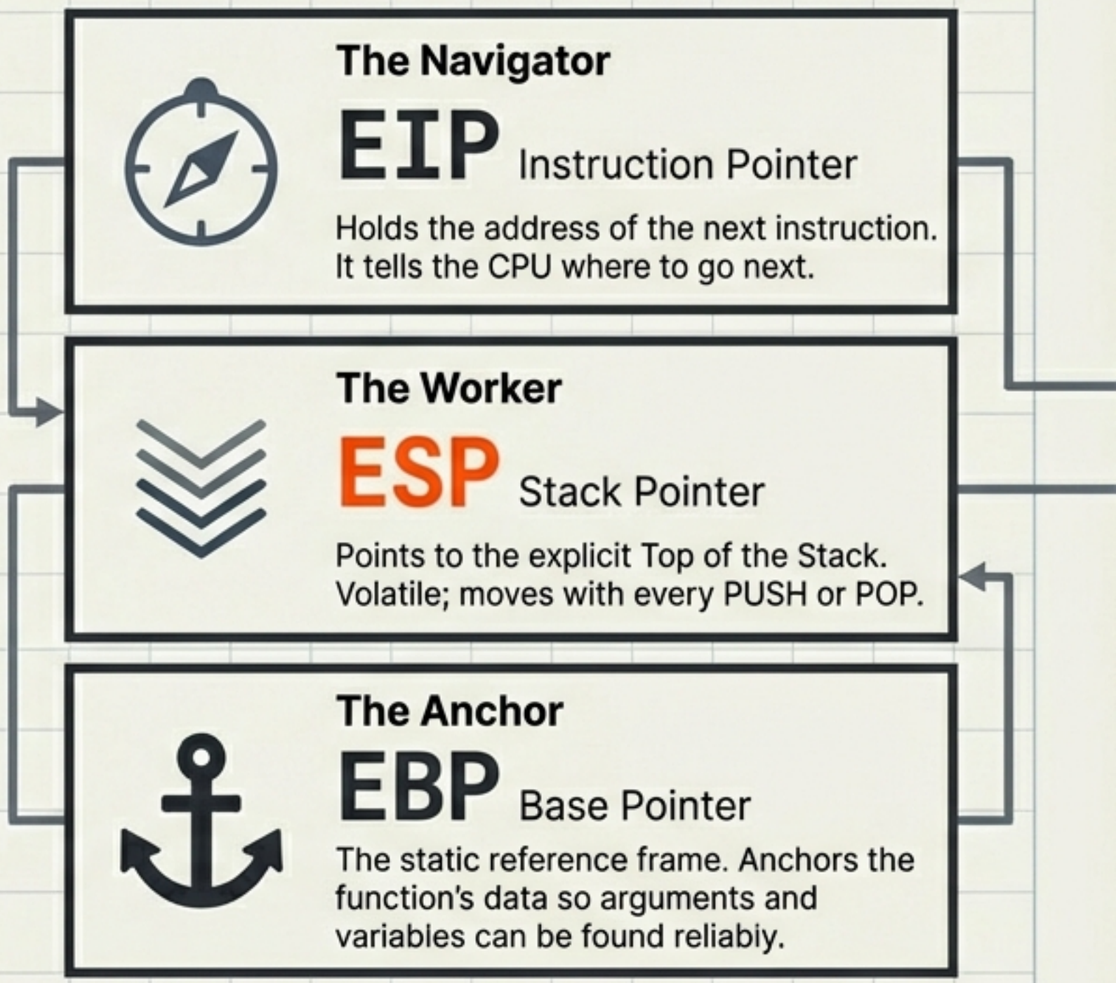
- High Addresses = Top of the diagram.
- Low Addre**sses** = Bottom of the diagram.



The Registers: Actors on the Stage

Virtual memory is a linear address space. In x86 architecture, the Stack and Heap grow toward each other to maximize available space.

- High Addresses = Top of the diagram.
- Low Addre**sses** = Bottom of the diagram.



Note: In 64-bit systems, 'E' becomes 'R' (e.g., RIP, RSP, RBP).

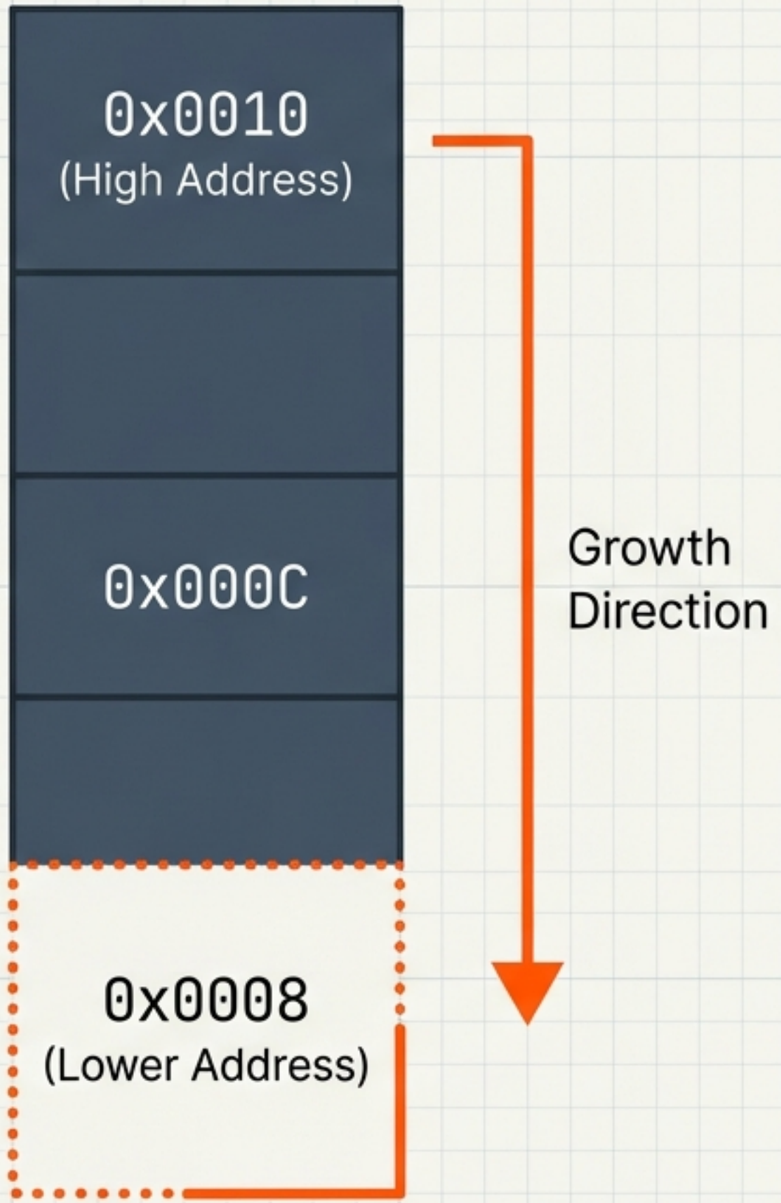
The Golden Rule: Growing Down

PUSH = ESP - 4

POP = ESP + 4

The Stack grows towards lower memory addresses. To add data, we must subtract from the address pointer.

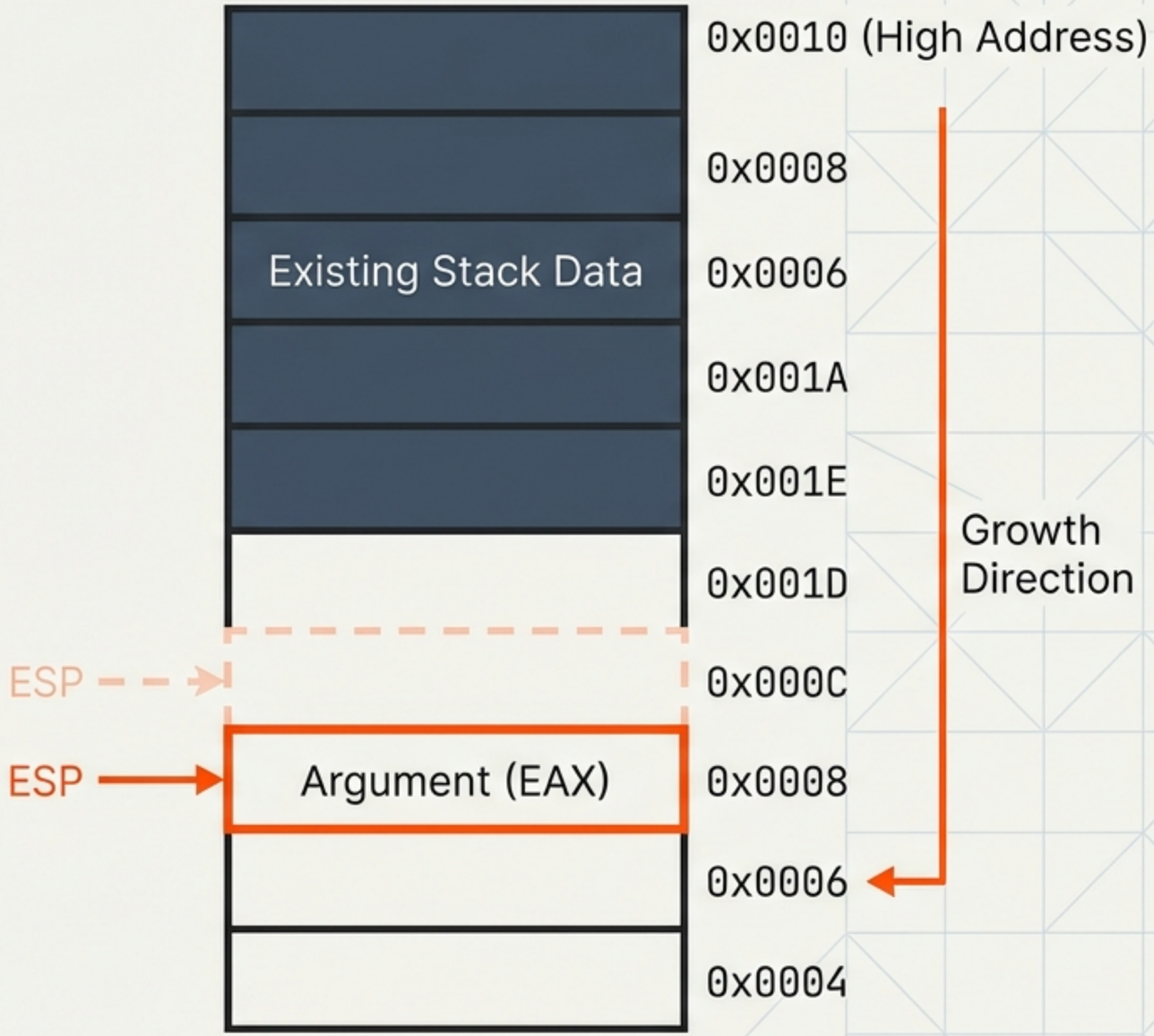
Think of it as digging a hole deeper into the ground (lower numbers).



Phase 1: The Setup

```
PUSH EAX
CALL foo
ADD  ESP, 4
```

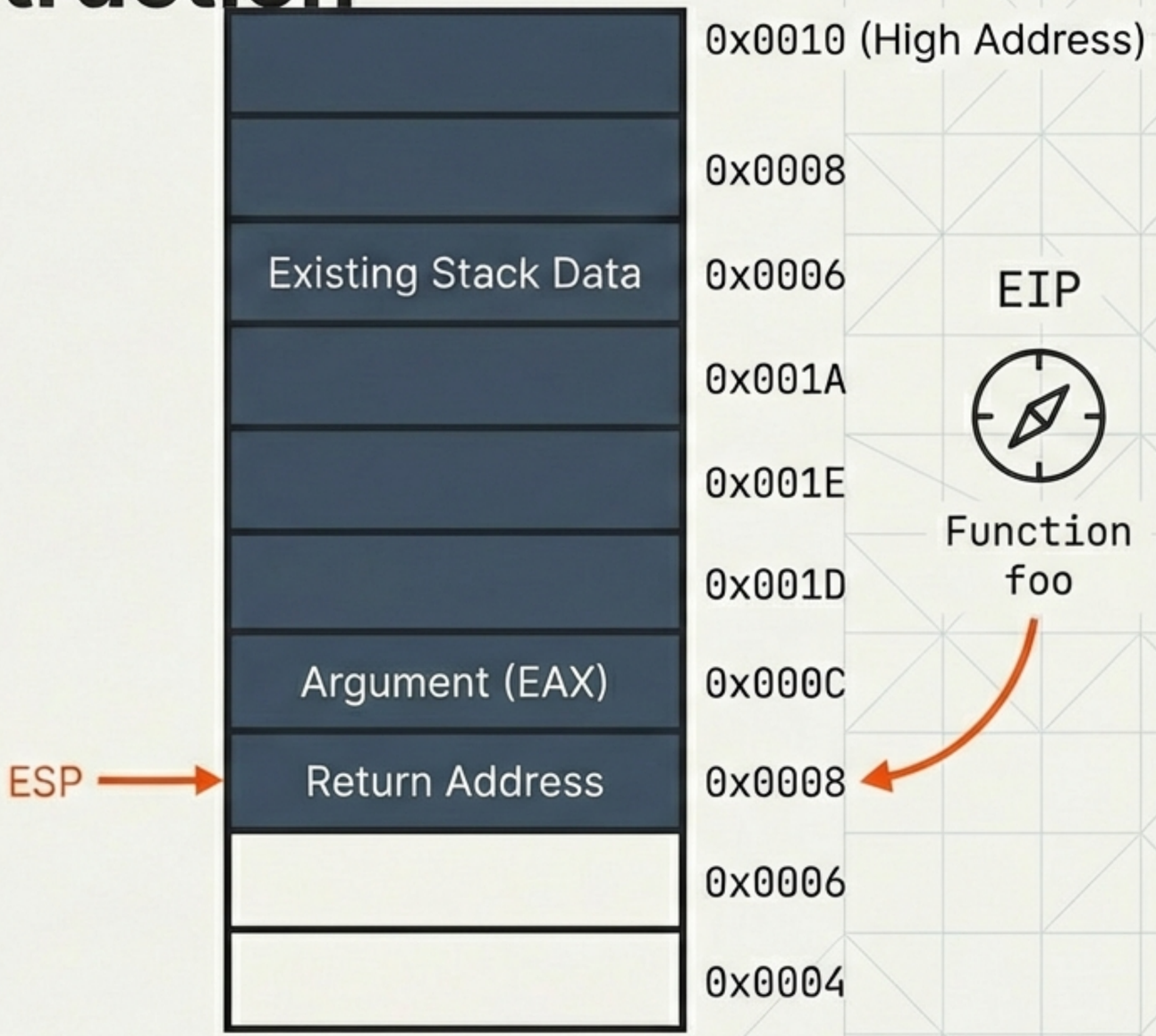
The Caller pushes arguments onto the stack. ESP decrements by 4 bytes.



Phase 2: The CALL Instruction

```
PUSH EAX
CALL foo
...
```

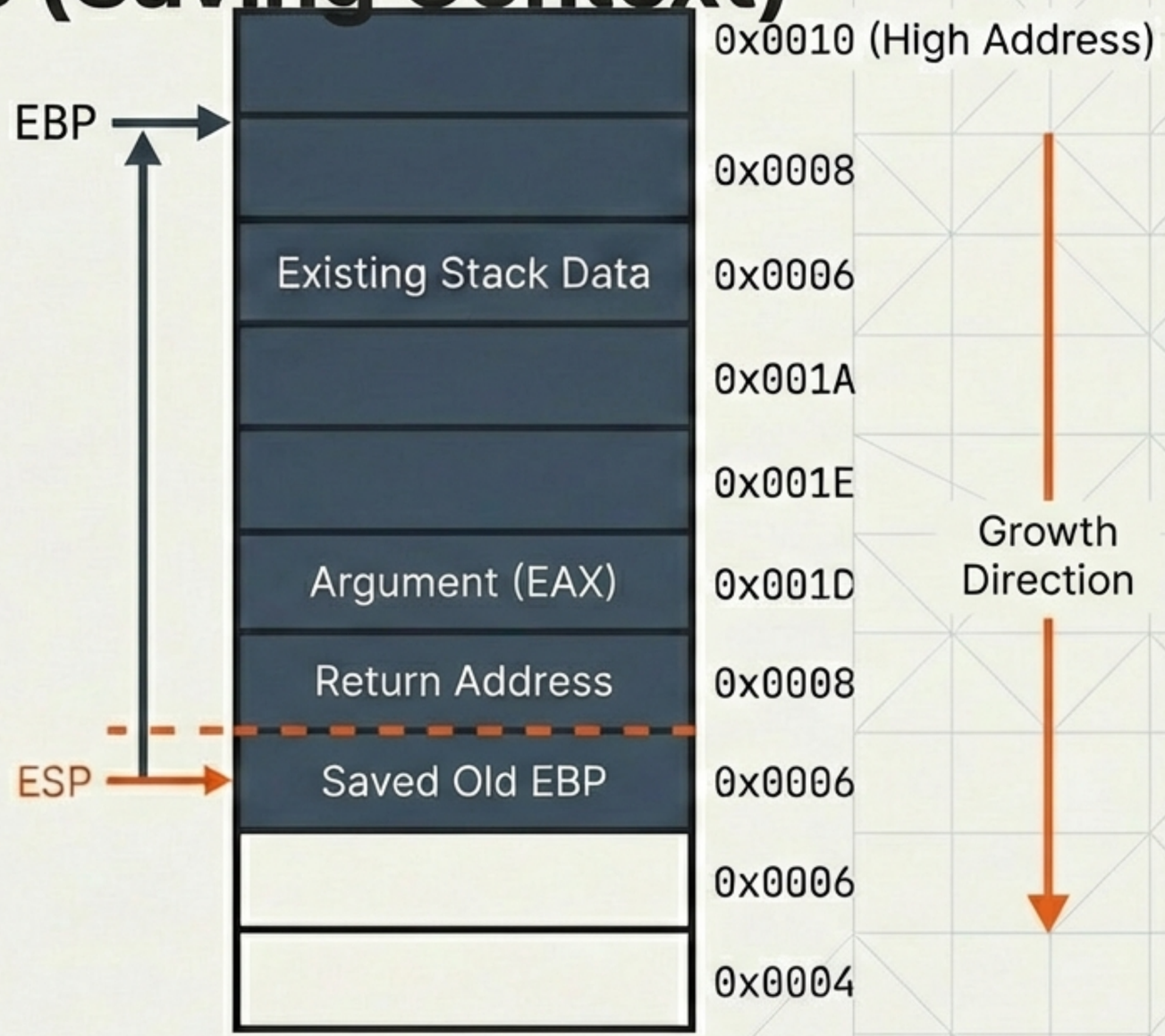
Hardware Interrupt: The CPU pushes the Return Address so it knows where to resume. EIP jumps to the function code.



Phase 3: The Prologue (Saving Context)

```
function foo:
PUSH EBP
MOV EBP, ESP
```

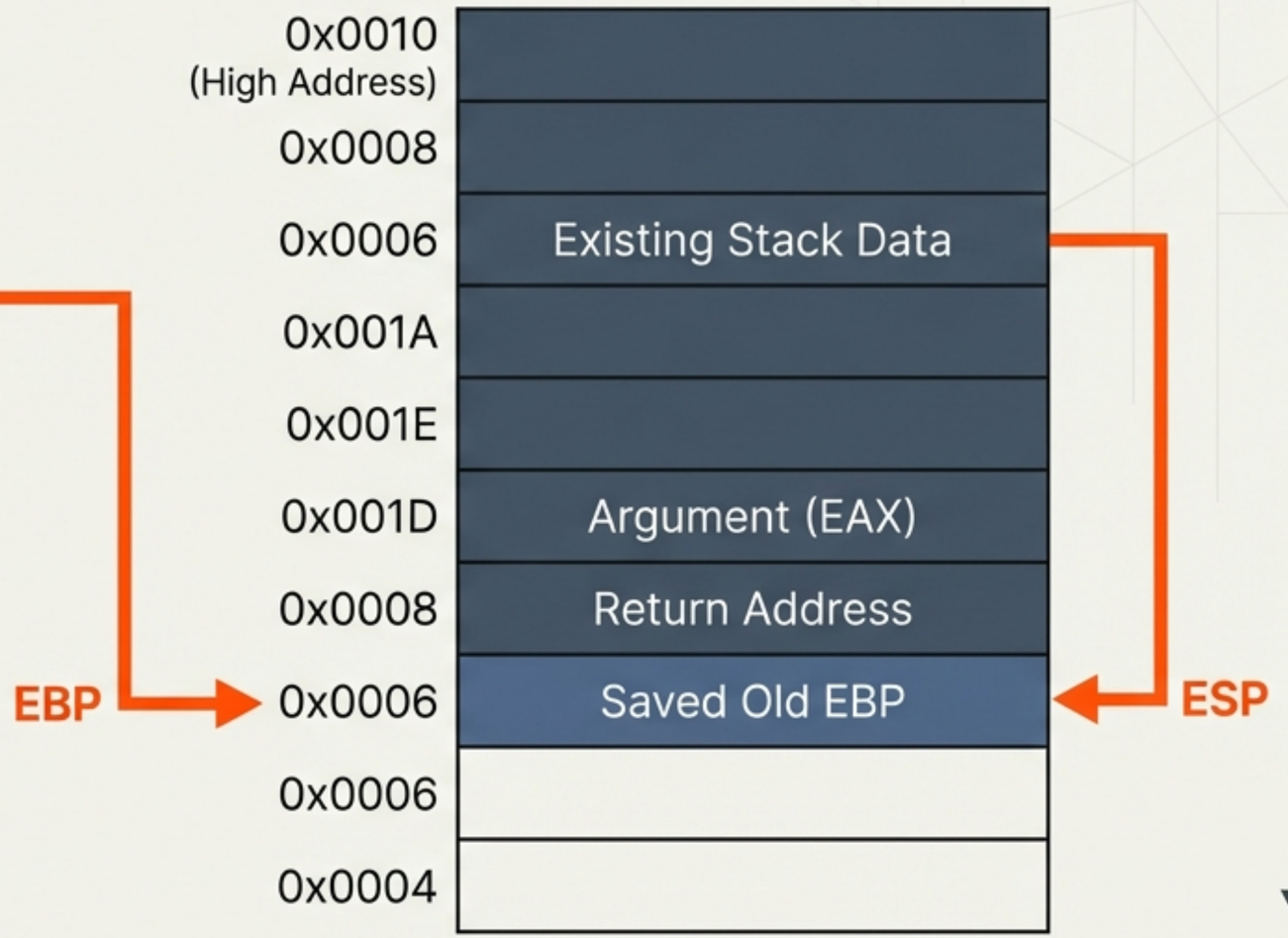
We enter the function. First, we save the Caller's Base Pointer so we can restore it later.



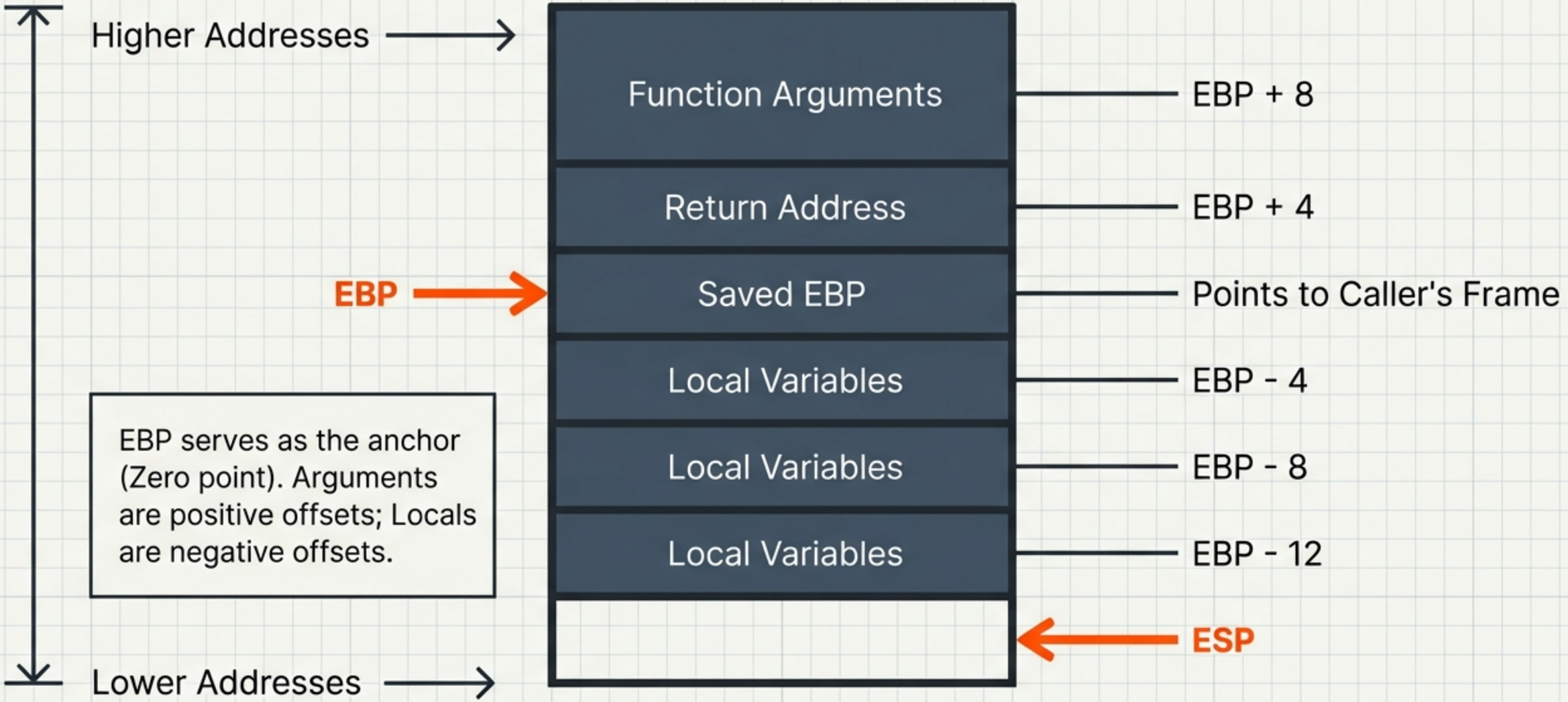
Phase 3: The Prologue (Setting the Anchor)

```
function foo:
    PUSH EBP
    MOV EBP, ESP
```

Snapshot. We lock the Base Pointer (EBP) to the current position. The new Stack Frame is established.



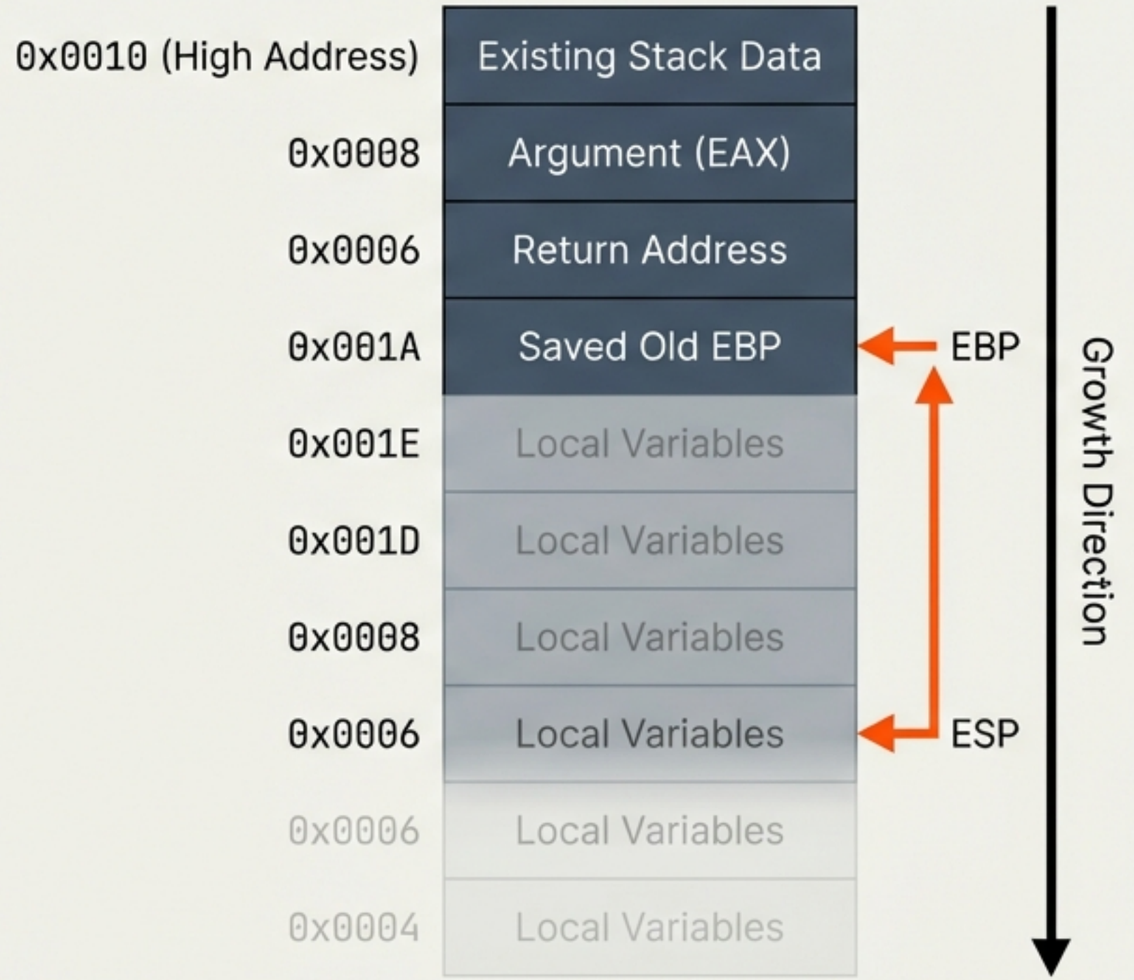
Anatomy of a Stack Frame



Phase 4: The Epilogue (Restoring the Stack)

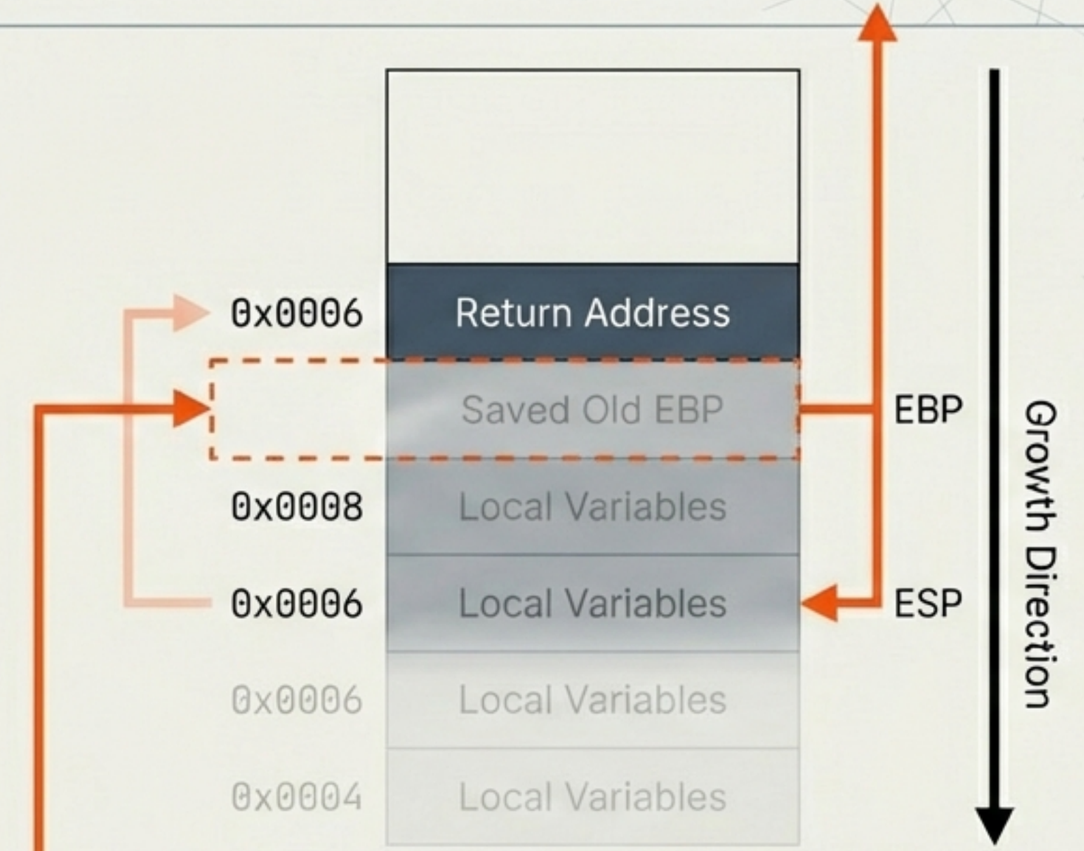
Teardown. We effectively delete all local variables instantly by snapping ESP back to the anchor (EBP).

```
...
MOV ESP, EBP
POP EBP
```



Phase 4: The Epilogue (Restoring the Anchor)

```
MOV ESP, EBP
POP EBP
RET
```

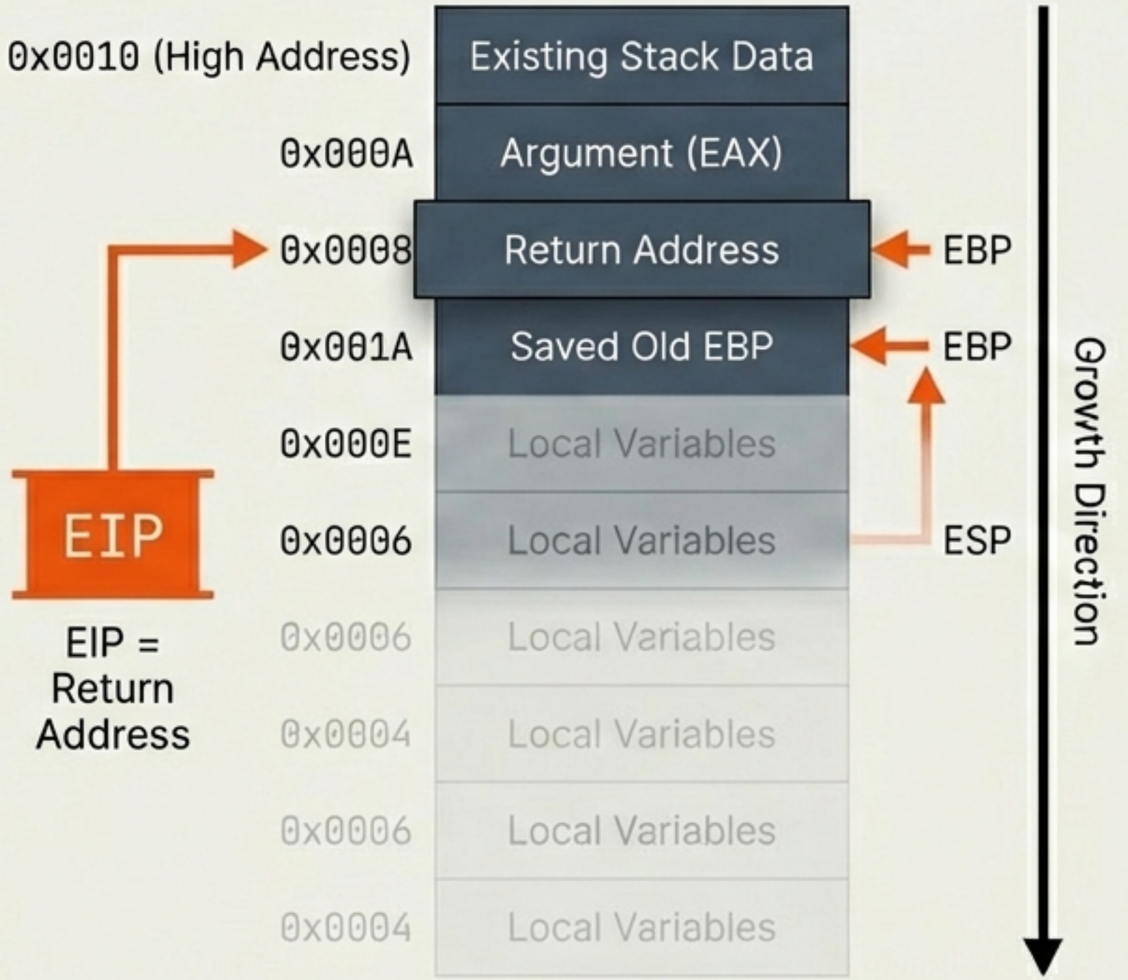


The Saved EBP is popped back into the register. The Anchor is restored to the Caller's frame. ESP points to the Return Address.

Phase 5: The Return

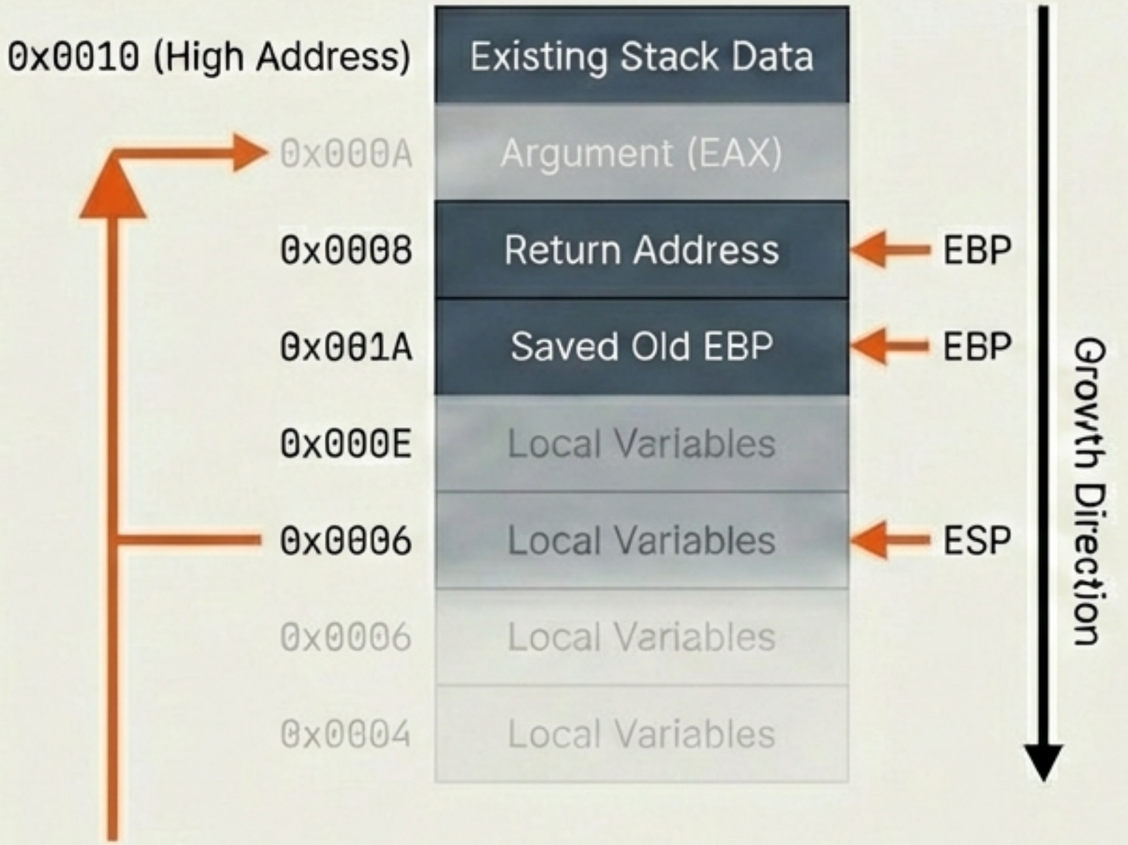
Control flow transfer. The CPU pops the Return Address into EIP and jumps back to the caller.

```
MOV ESP, EBP
POP EBP
RET
```



Phase 6: Caller Cleanup

```
CALL foo
ADD ESP, 4
```



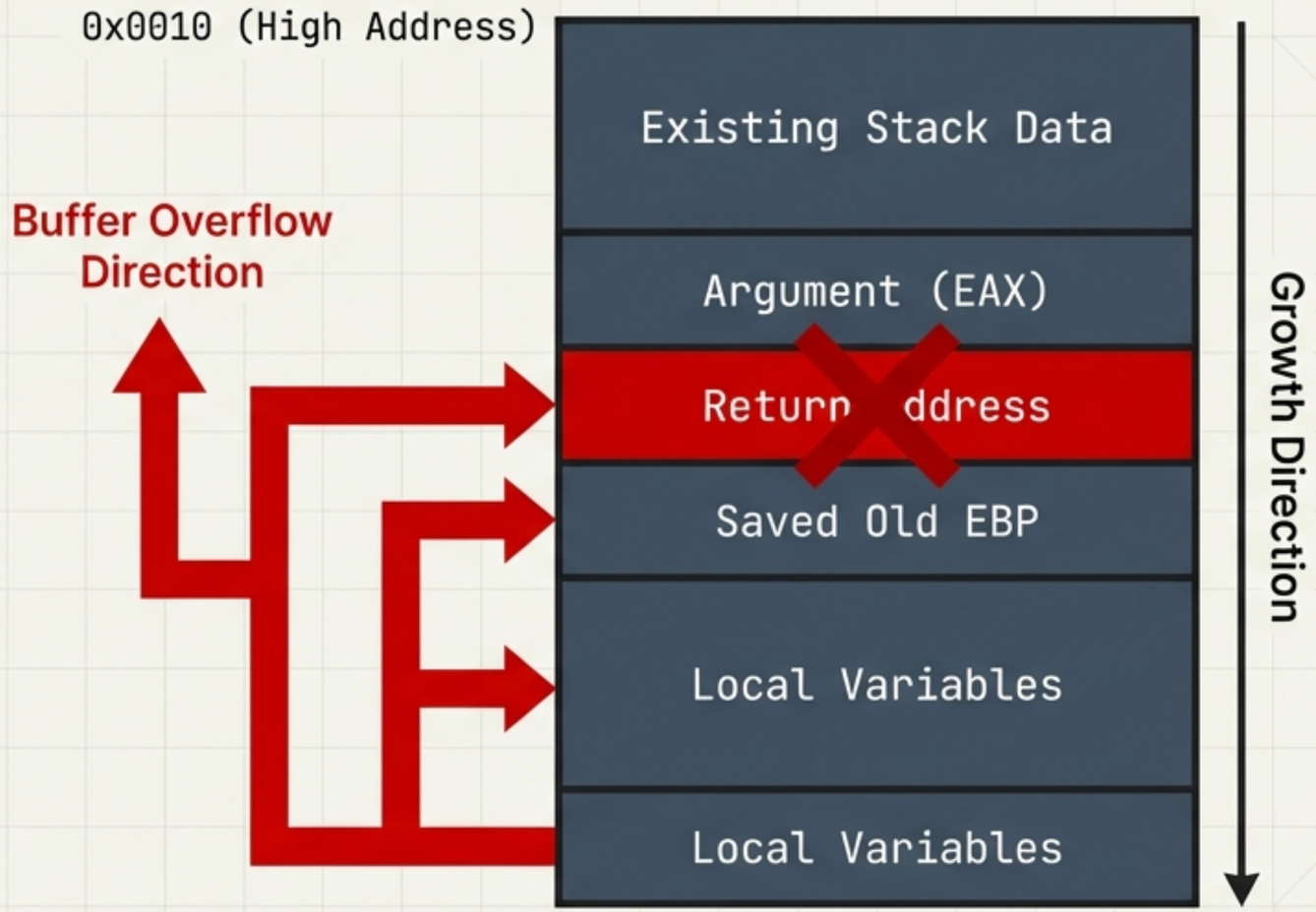
Calling Convention (cdecl). The Caller must remove the arguments it pushed. Stack is fully restored.

Security: The Cost of Misalignment

If a local buffer overflows, it writes to higher addresses.

Result: The Return Address is overwritten.

Consequence: Control Flow Hijacking or SegFault.



Core Takeaways

Direction: Stack grows **Down** (High -> Low).

Roles: ESP tracks the Top; EBP anchors the Frame

Lifecycle: Prologue builds the frame; Epilogue destroys it.

Precision: **1 byte** of misalignment crashes the program.